

# Taller práctico: entendiendo algoritmos con Python

**Objetivo:** aprender los conceptos de la sesión mediante un solo problema sencillo.

Vamos a trabajar con una lista de números que representa las **notas de estudiantes**.

Durante el ejercicio vamos a practicar:

1. Complejidad  $O(n)$
2. Complejidad  $O(n^2)$
3. **Búsqueda lineal**
4. **Búsqueda binaria**
5. **Recursividad y recurrencias**
6. **Divide y vencerás**
7. **Merge Sort**
8. **Quick Sort**

🌟 La idea no es hacer un programa complicado. Lo importante es observar **qué hace el algoritmo y cómo cambia su trabajo cuando aumenta la cantidad de datos**.

## 1. Datos iniciales

Tenemos las siguientes notas:

[75, 60, 90, 85, 70, 95, 55, 80]

Primero vamos a crear nuestra lista en Python.

```
notas = [75, 60, 90, 85, 70, 95, 55, 80]

print("Notas:", notas)
print("cantidad de estudiantes:", len(notas))

Notas: [75, 60, 90, 85, 70, 95, 55, 80]
Cantidad de estudiantes: 8
```

## 2. $O(n)$ : recorrer una lista

Definición

Un algoritmo es  $O(n)$  cuando su trabajo crece proporcionalmente con la cantidad de elementos.

Aquí vamos a recorrer la lista una sola vez para contar cuántos estudiantes aprobaron.

Supongamos que se aprueba con una nota de **60 o más**.

```
aprobados = 0

for nota in notas:
    if nota >= 60:
        aprobados += 1

print("Estudiantes aprobados:", aprobados)
```

### 🔗 Pregunta para la clase

¿Cuántas veces se ejecuta el `for` si tenemos:

- 8 estudiantes?
- 100 estudiantes?
- 1.000 estudiantes?

🌟 La respuesta crece con  $n$ , por eso este recorrido es  $O(n)$ .

## 3. $O(n^2)$ : comparar todos contra todos

Ahora queremos encontrar **pares de estudiantes que tengan exactamente la misma nota**.

Para hacerlo de manera sencilla, vamos a comparar cada nota con todas las demás.

Esto requiere dos ciclos.

```
for i in range(len(notas)):
    for j in range(i + 1, len(notas)):
        if notas[i] == notas[j]:
            print("Misma nota:", notas[i])
```

### 🔗 ¿Por qué $O(n^2)$ ?

Tenemos un ciclo dentro de otro.

De manera aproximada:

$n \times n = n^2$

Por eso este tipo de solución tiene una complejidad  $O(n^2)$ .

🔗 **Pregunta:** ¿qué creen que pasaría si en lugar de 8 estudiantes tuviéramos 1.000?

## 4. 🌀 Búsqueda lineal

Definición

La **búsqueda lineal** revisa los elementos **uno por uno** hasta encontrar el dato.

Vamos a buscar la nota **95**.

```
objetivo = 95
comparaciones = 0
encontrado = False

for nota in notas:
    comparaciones += 1

    if nota == objetivo:
        encontrado = True
        break

print("¿Encontrado?", encontrado)
print("Comparaciones realizadas:", comparaciones)
```

Complejidad

La búsqueda lineal es  $O(n)$  porque, en el peor caso, podemos tener que revisar todos los elementos.

🔗 **Pregunta:** ¿qué ocurre si buscamos un valor que no existe en la lista?

## 5. 🚀 Búsqueda binaria

Definición

La **búsqueda binaria** busca un elemento descartando aproximadamente **la mitad de los datos en cada paso**.

🚩 **Condición indispensable:** la lista debe estar **ordenada**.

Primero ordenamos las notas.

```
notas_ordenadas = sorted(notas)
print(notas_ordenadas)
```

Ahora vamos a programar nuestra propia búsqueda binaria.

La idea es:

1. Mirar el elemento del centro.
2. Si es el que buscamos  $\rightarrow$  terminamos.
3. Si el objetivo es mayor  $\rightarrow$  buscamos en la mitad derecha.
4. Si es menor  $\rightarrow$  buscamos en la mitad izquierda.
5. Repetimos.

```
def busqueda_binaria(lista, objetivo):
    izquierda = 0
    derecha = len(lista) - 1
    comparaciones = 0

    while izquierda <= derecha:
        medio = (izquierda + derecha) // 2
        comparaciones += 1

        if lista[medio] == objetivo:
            return medio, comparaciones
        elif lista[medio] < objetivo:
            izquierda = medio + 1
        else:
            derecha = medio - 1

    return -1, comparaciones

posicion, comparaciones = busqueda_binaria(notas_ordenadas, 95)

print("Posición:", posicion)
print("Comparaciones:", comparaciones)
```

### 🔗 Comparación

- Búsqueda lineal  $\rightarrow O(n)$
- Búsqueda binaria  $\rightarrow O(\log n)$

La ventaja de la binaria se vuelve mucho más evidente cuando la lista tiene muchos elementos.

🌟 **Importante:** si la lista no está ordenada, no podemos aplicar directamente esta búsqueda.

## 6. 🧩 Recursividad y recurrencias

Definición

Un algoritmo es **recursivo** cuando una función se llama a sí misma para resolver un problema más pequeño.

Vamos a crear una función muy sencilla que sume los números desde **1** hasta **n**.

Ejemplo:

suma(4) = 4 + 3 + 2 + 1

La función se va reduciendo hasta llegar al caso base.

```
def suma_recursiva(n):
    # Caso base
    if n == 1:
        return 1

    # Problema más pequeño
    return n + suma_recursiva(n - 1)

print(suma_recursiva(5))
```

¿Dónde está la recurrencia?

Podemos expresar la función como:

$T(n) = T(n - 1) + O(1)$

Esto significa, de manera sencilla:

Para resolver el problema de tamaño  $n$ , resolvemos primero un problema de tamaño  $n-1$  y hacemos una operación adicional.

No vamos a resolver matemáticamente esta recurrencia; aquí interesa reconocer **qué significa una recurrencia**.

## 7. 🚀 Divide y vencerás

Definición

**Divide y vencerás** es una estrategia que:

**DIVIDE  $\rightarrow$  RESUELVE  $\rightarrow$  COMBINA**

Vamos a aplicarla para encontrar el número máximo de una lista.

Por ejemplo:

```
[75, 60, 90, 85]

↓ dividir

[75, 60] [90, 85]

↓ resolver

75      90

↓ combinar

90
```

La idea importante es que el problema grande se convierte en problemas más pequeños.

```
def maximo_divide_venceras(lista):
    # Caso base
    if len(lista) == 1:
        return lista[0]

    # DIVIDIR
    medio = len(lista) // 2
    izquierda = lista[:medio]
    derecha = lista[medio:]

    # RESOLVER
    max_izquierda = maximo_divide_venceras(izquierda)
    max_derecha = maximo_divide_venceras(derecha)

    # COMBINAR
    return max(max_izquierda, max_derecha)

print("Máximo:", maximo_divide_venceras(notas))
```

## 8. 🌿 Merge Sort

Definición

**Merge Sort** es un algoritmo de ordenamiento que utiliza **divide y vencerás**.

Sus pasos son:

1. Dividir la lista.
2. Ordenar las partes.
3. Combinar las partes ordenadas.

Vamos a observar una implementación sencilla.

```
def merge_sort(lista):
    # Caso base
    if len(lista) <= 1:
        return lista

    # DIVIDIR
    medio = len(lista) // 2
    izquierda = merge_sort(lista[:medio])
    derecha = merge_sort(lista[medio:])

    # COMBINAR
    resultado = []
    i = 0
    j = 0

    while i < len(izquierda) and j < len(derecha):
        if izquierda[i] < derecha[j]:
            resultado.append(izquierda[i])
            i += 1
        else:
            resultado.append(derecha[j])
            j += 1

    resultado.extend(izquierda[i:])
    resultado.extend(derecha[j:])

    return resultado

print("Original:", notas)
print("Ordenada:", merge_sort(notas))
```

### 🌟 Complejidad de Merge Sort

$O(n \log n)$

¿Por qué aparece  $\log n$ ?

Porque la lista se va dividiendo aproximadamente por la mitad.

¿Por qué aparece  $n$ ?

Porque al combinar las partes hay que procesar los elementos.

Por eso:

**Merge Sort  $\rightarrow O(n \log n)$**

## 9. ⚡ Quick Sort

Definición

**Quick Sort** es un algoritmo de ordenamiento que utiliza **divide y vencerás** y selecciona un elemento llamado **pivote**.

Vamos a usar el **último elemento como pivote**, igual que en el ejercicio de la guía.

```
def quick_sort(lista):
    if len(lista) <= 1:
        return lista

    # Último elemento como pivote
    pivote = lista[-1]

    menores = []
    mayores = []

    for elemento in lista[:-1]:
        if elemento <= pivote:
            menores.append(elemento)
        else:
            mayores.append(elemento)

    return quick_sort(menores) + [pivote] + quick_sort(mayores)

print("Original:", notas)
print("Ordenada:", quick_sort(notas))
```

### 🔗 ¿Qué está pasando?

Si tenemos:

[8, 3, 5, 1, 9, 2]

El último elemento es:

pivote = 2

Separamos:

| menores |  | pivote |  | mayores |
|---------|--|--------|--|---------|
| 1       |  | 2      |  | 8,3,5,9 |

Después repetimos el proceso con las partes.

Complejidad

- Promedio  $\rightarrow O(n \log n)$
- Peor caso  $\rightarrow O(n^2)$

El peor caso ocurre cuando las divisiones quedan muy desbalanceadas.

## 🎯 Reto final

Ahora intenta modificar el programa sin mirar las soluciones anteriores.

Reto 1

Crea una función `contar_aprobados(lista)` que recorra una lista y devuelva cuántos estudiantes aprobaron.

**Pregunta:** ¿qué complejidad tiene?

Reto 2

Crea una función que encuentre cuántas veces aparece una nota determinada comparando todos los elementos.

**Pregunta:** ¿puedes hacerla usando dos ciclos? ¿Qué complejidad tendría?

Reto 3

Busca una nota usando:

- búsqueda lineal
- búsqueda binaria

**Pregunta:** ¿qué diferencia observas?

Reto 4

Modifica `quick_sort()` para que muestre cuál es el pivote en cada llamada.

Reto 5

Explica con tus propias palabras:

¿Cuál es la diferencia entre  $O(n)$ ,  $O(n^2)$  y  $O(\log n)$ ?

### 🔗 Cierre: mapa mental

| Tema              | Idea clave                                                      |
|-------------------|-----------------------------------------------------------------|
| Big-O             | Cómo crece el costo del algoritmo                               |
| $O(n)$            | Recorremos aproximadamente $n$ elementos                        |
| $O(n^2)$          | Dos recorridos anidados / comparaciones entre elementos         |
| Recurrencia       | Describe el costo de un algoritmo recursivo                     |
| Divide y vencerás | Dividir $\rightarrow$ resolver $\rightarrow$ combinar           |
| Merge Sort        | Divide y vencerás + combinar $\rightarrow O(n \log n)$          |
| Quick Sort        | Divide y vencerás + pivote $\rightarrow$ promedio $O(n \log n)$ |
| Búsqueda lineal   | Uno por uno $\rightarrow O(n)$                                  |
| Búsqueda binaria  | Mitad de la mitad $\rightarrow O(\log n)$ , con lista ordenada  |

### 🔗 Pregunta final para los estudiantes

Si tuvieras 1.000.000 de datos, ¿qué algoritmo o estrategia preferirías y por qué?

No busques solamente que digan una respuesta: pídeles que la **justifiquen usando la complejidad**.